

Thus far you have seen the concept of smoothing a 1D or 2D function, via convolution by a filter or a kernel. Often that kernel is taken to have the shape of a Gaussian. We shall begin by reviewing some of the key concepts we have already covered and then indicate what is special about using a Gaussian. We shall then describe the process of generating a Gaussian scale space and then relate this process to a type of isotropic diffusion, given by the heat equation. It turns out that the heat equation point of view allows us to recast the process of Gaussian smoothing as a simple iterative one, via the discretization of a simple differential equation. In that equation, the notion of the Laplacian, which you have already seen in the context of edge detection (Marr and Hildreth), shows up. We shall later describe a generalization of isotropic diffusion to anisotropic diffusion, focussing on the Perona-Malik equation. Here the essential heuristic idea is to smooth while preserving edges, by slowing down the diffusion at spatial locations where the magnitude of the image gradient is high. We shall then describe how the concept of a scale space, combined with downsampling and upsampling lead to image pyramids. In what follows, we shall use the terms “Gaussian smoothing” and “Gaussian convolution” to mean the same thing.

Review

Recall that convolution of an image $I(x, y)$ by a kernel $f(x, y)$ in 2D is given by

$$f(x, y) * I(x, y) = \sum_{u, v} f(x - u, y - v) I(u, v).$$

We often use a zero mean Gaussian for the kernel, which is given by

$$G(x, y, \sigma) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2 + y^2}{2\sigma^2}}.$$

Recall from an earlier lecture that this expression follows from the fact that a 2D Gaussian can be written as the product of two 1D Gaussians, with each having the same standard deviation σ .

Why do we smooth with a Gaussian? One reason that we talked about is the problem of edge detection where edges in $I(x, y)$ are associated with spatial locations having a large value of $\|\nabla I(x, y)\|$, the magnitude of the image gradient. In order to compute the gradient we must assume that the image is differentiable, thus smoothing it to make it so is often the first step.

We also saw in an earlier lecture that Gaussian convolution, being a linear operation, allows for derivatives of Gaussian smoothed images to be efficiently computed. The basic idea is to directly convolve the image with a derivative of (or higher order derivatives of) a Gaussian. For example, in the 1D case,

$$\frac{d}{dx}(G(x, \sigma) * I(x)) = \frac{d}{dx}(G(x, \sigma) * I(x)).$$

Scale Space

In addition to separability, it turns out that convolution with a Gaussian also satisfies a semi-group property. To illustrate this, let $\sigma_3 = \sigma_2 + \sigma_1$. Then we have that

$$G(x, y, \sigma_3) * I(x, y) = G(x, y, \sigma_2) * (G(x, y, \sigma_1) * I(x, y)).$$

By virtue of this property, convolution with a Gaussian having a large standard deviation can be carried out by successive convolution with Gaussians having a smaller standard deviation. The latter strategy is more efficient in terms of storage requirements.

This idea of generating higher order derivative of Gaussian smoothed images is a foundational concept in scale space theory, and for the interested reader there is an excellent account of this subject in the book “Front-End Vision and Multiscale Image Analysis” by Bart M. Haar Romeny, along with many examples and visualizations developed using Mathematica. It turns out that if one chooses to satisfy certain axioms (linearity, the semi-group property, causality) the Gaussian can be shown to be the unique smoothing kernel.

We are now ready to define a Gaussian scale space, which is essentially a family of images generated by smoothing an original image $I(x, y)$ with Gaussians having larger and larger σ . This results in a stack of increasingly blurred images, where the σ associated with any level in the stack is its scale parameter. As the images become increasingly blurred one goes from fine to coarser scales. In the literature this process of Gaussian smoothing is often combined with tracking features in scale space (typically local extrema of image intensity or the zero-crossings associated with convolution with the Laplacian of a Gaussian, as in the Marr and Hildreth paper). In fact, in a classic paper “Reconstructions from Zero Crossings in Scale Space”, Hummel showed that in a family of smoothed images generated by convolving with a Laplacian of a Gaussian having increasing σ , it is possible to reconstruct the original images from the blurred ones, if the zero-crossings along with gradient information along them, is included.

Connection to the Heat Equation

Hummel also observed that the Gaussian is the kernel of the isotropic heat equation, and thus smoothing with a Gaussian is equivalent to running the heat equation, also known as isotropic diffusion or Laplace’s equation, for a certain amount of time. In more detail, let the family of smoothed images be parametrized by t , with $t > 0$. Then the family $I(x, y, t)$ is obtained by solving Laplace’s equation, which is the partial differential equation

$$\frac{\partial I(x, y, t)}{\partial t} = \Delta I(x, y, t) = \frac{\partial^2 I(x, y, t)}{\partial x^2} + \frac{\partial^2 I(x, y, t)}{\partial y^2}.$$

In fact, convolving with a Gaussian having standard deviation σ is equivalent to running the above heat equation for a total time given by $1/2\sigma^2$, and this result extends to images (and correspondingly Gaussians) of higher dimensions as well. As such, it is possible to very efficiently create Gaussian smoothed images, by not doing convolution at all but rather just running a simple iterative process. In class we saw a demonstration of heat equation run on an image.

Perona-Malik Anisotropic Diffusion

Due to the fact that $\Delta I(x, y, t) = \text{div}(\nabla I(x, y, t))$, the heat equation can also be written as

$$\frac{\partial I(x, y, t)}{\partial t} = \text{div}(\nabla I(x, y, t)).$$

As we observed in class, since it is equivalent to smoothing with a Gaussian, running the heat equation will create increasingly blurred versions of an original input image. An issue with this

is that salient features, such as edges, will become diffuse following which it will not be easy to localize them. Observing this, Perona and Malik (IEEE TPAMI, 1990) suggested a modification of the heat equation, where the basic idea was to slow down the diffusion or the amount of blurring, in the vicinity of a potential edge. Since one has the chicken-and-egg problem of locating the edge, Perona and Malik used a heuristic where a diffusion coefficient $c(x, y, t)$ was introduced into the heat equation, and which was taken to be a monotonically decreasing function of the magnitude of the image gradient, i.e., $c(x, y, t) \propto \|\nabla I(x, y, t)\|^{-1}$. The Perona-Malik diffusion equation is closely related to the heat equation, and is given by

$$\frac{\partial I(x, y, t)}{\partial t} = \text{div}(c(x, y, t) \nabla I(x, y, t)) = c(x, y, t) \Delta I(x, y, t) + \nabla c(x, y, t) \cdot \nabla I(x, y, t).$$

Observe that with $c(x, y, t) = 1$, we have the original heat equation, so the Perona-Malik equation includes isotropic diffusion or Gaussian smoothing, as a special case. Further, whereas we think of $c(x, y, t)$ as varying with position and time in the scale space, in practice Perona and Malik suggested two particular choices for the diffusion coefficient, namely

$$c(\|\nabla I\|) = \frac{1}{1 + \left(\frac{\|\nabla I\|}{K}\right)^2}$$

and

$$c(\|\nabla I\|) = e^{-\left(\frac{\|\nabla I\|}{K}\right)^2},$$

where K is a constant chosen to control the sensitivity to image gradient strength. There is no particular justification for these precise choices of monotonically decreasing functions of image gradient magnitude. However, both share the property that they are close to 1 when the image gradient magnitude is small and then decrease to 0 as the gradient magnitude increases. In class we shall show an example of running the Perona-Malik diffusion equation and will contrast this with the heat equation.

Gaussian Pyramids and Laplacian Pyramids

Often in computer vision tasks such as image registration, stereo matching or object tracking, topics we will cover later in the course, it is necessary for algorithms to be efficient to achieve near real-time performance. In such situations it can be helpful to reduce computation by generating coarser and smaller versions of an input image, and then refining estimates obtained from the coarser versions when necessary. Two popular examples are Gaussian and Laplacian pyramids.

The idea behind a Gaussian pyramid is very similar to that behind a Gaussian scale space, with the addition that with each smoothing level one also introduces a decimation or sampling step to generate smaller versions. One can think of this as convolution followed by sampling. For example, in the Gaussian pyramid shown in Fig. 1 (top), each level is created by smoothing the previous one by convolving with a Gaussian, and then sampling every second pixel in each direction.

One might reasonably ask how one would attempt to recover a lower level in the pyramid from the next higher level. For this an interpolation process is required. The restored level can then be compared with the original level, to assess their difference; in general some detail will be lost. When a Gaussian pyramid is involved, the difference between a level and the reconstruction of that level

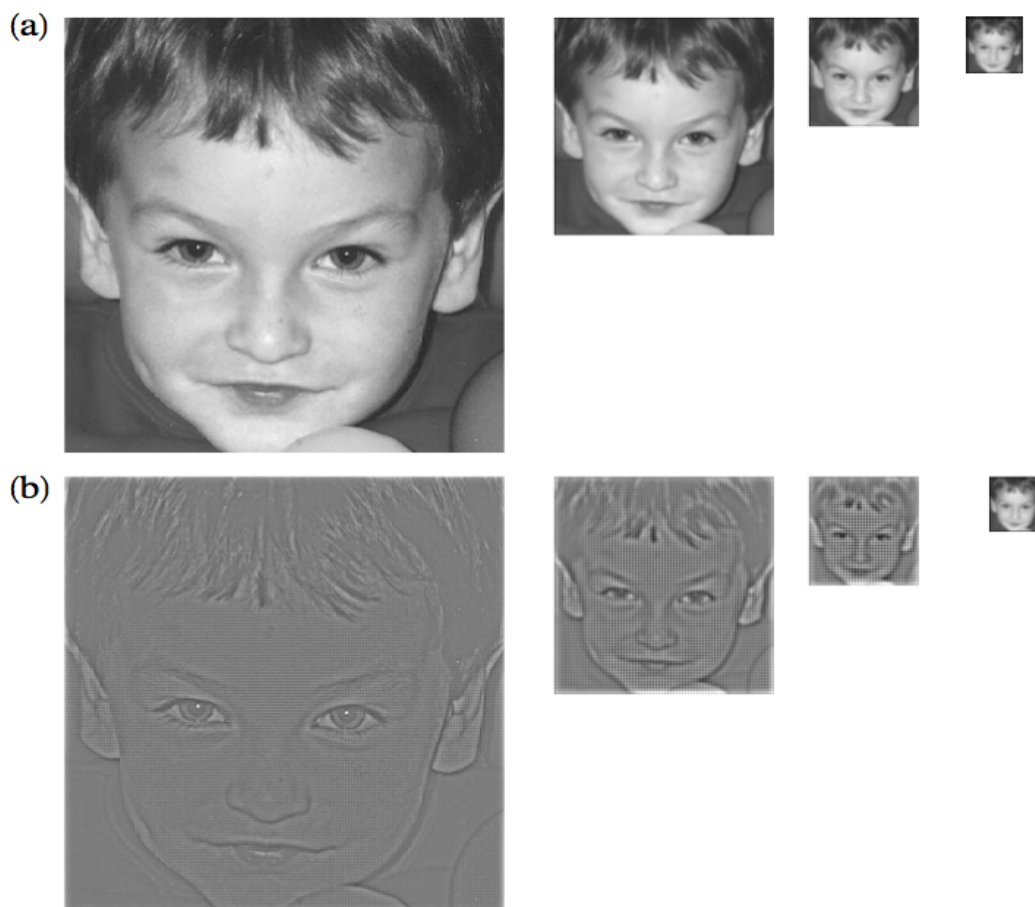


Figure 1: (This example was taken from Brian Wandell's Foundations of Vision (<https://foundationsofvision.stanford.edu/chapter-8-multiresolution-image-representations>)). Top: A Gaussian pyramid, where each level is generated from the previous one by convolving with a Gaussian, and then sub-sampling. Here the decimation at each level is by a factor of 2 in each direction, so that each new level is one quarter the size of the previous level. Bottom: The associated Laplacian pyramid, which is the difference between a level and its reconstruction from the next higher level by interpolation and upsampling.

from the next higher level by interpolation, leads to a Laplacian pyramid. This is illustrated in Fig. 1 (bottom). Observe that the differences highlight edge like features at each level. This should not be surprising because it is precisely these features that are most affected by Gaussian blurring.